SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 10%

QUESTIONS: Total Marks: 50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Santhosh Kumar /192411015 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:V.Santhosh Kumar

Annexure A
DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

t1 = x + y

t2 = t1 * z

t3 = t2 / w

t4 = t3 - p

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Answer:

• Sequence Analysis: The four instructions form a linear data-dependency chain: t1 depends on x and y; t2 depends on t1 and z; t3 depends on t2 and w; t4 depends on t3 and p, so each temporary carries forward the result of the previous computation.

• Expression Tree: The equivalent expression tree is built bottom-up: a '+' node with leaves x and y (giving t1); this node becomes the left child of a '*' node with leaf z (giving t2); that becomes the left child of a '/' node with leaf w (giving t3); and finally that becomes the left child of a '-' node with leaf p (giving t4), so the tree reflects ((((x + y) * z) / w) - p).

• Role of Temporaries: Temporary variables (t1-t4) store intermediate results so each sub-expression is computed exactly once and can be reused or referenced by later instructions without re-evaluating the original expression.

• Benefit: This avoids repeated computation of shared sub-expressions, keeps each three-address instruction simple (at most one operator), and gives the compiler a clear, linear structure that is easy to translate directly into machine code or further optimize.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Answer:

• Allocation Process: The compiler scans the symbol table and, for each variable, reserves a memory block equal to its declared datatype's size: p (char) = 1 byte, q (double) = 8 bytes, r (int) = 4 bytes, s (float) = 4 bytes.

• Total Memory: Total memory required = 1 + 8 + 4 + 4 = 17 bytes for the four variables, before any alignment padding is applied.

• Effect of Alignment: Because most processors require multi-byte data (like double and int) to start at addresses that are multiples of their size, the compiler inserts padding bytes between variables of different sizes, so declaring p (1 byte) immediately before q (8 bytes) would add up to 7 padding bytes to align q correctly.

• Memory Organization: Ordering variables from largest to smallest (q, then r/s, then p) minimizes padding and total allocated memory, so both datatype size and alignment rules together determine the compiler's actual memory layout, not just the sum of individual variable sizes.

Q3. Optimization in Intermediate Code

Given the intermediate code:

t1 = m * n

t2 = p + q

t3 = m * n

t4 = t2 - t3

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Answer:

• Step-by-Step Trace: t1 = m * n computes a new value; t2 = p + q computes a new value; t3 = m * n recomputes the exact same expression as t1; t4 = t2 - t3 uses the (redundant) t3.

• Optimization Opportunity: Common Subexpression Elimination: Since t3 = m * n is identical to the already-computed t1 = m * n, and neither m nor n changes in between, t3 can be eliminated and replaced by reusing t1, i.e., t4 = t2 - t1.

• Optimization Opportunity: Redundant Computation Removal: Removing the duplicate multiplication avoids recalculating a value the compiler already has, directly reducing the number of executed instructions from four to three.

• Improvement in Efficiency: Eliminating redundant computations reduces CPU cycles, lowers register or memory pressure from unnecessary temporaries, and produces more compact intermediate code, which speeds up both compilation and final program execution.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

if (x > y || p < q)

Intermediate code:

if x > y goto L1

if p < q goto L1

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Answer:

• Control Flow Interpretation: The intermediate code tests x > y first; if true, control jumps directly to L1 (the true branch). If false, it tests p < q; if that is true, control also jumps to L1. Only if both conditions are false does control fall through to goto L2 (the false branch).

• Mapping to the Boolean Expression: This directly implements the OR operation (x > y || p < q): the overall expression is true if either sub-condition is true, so a jump to L1 occurs as soon as any one condition succeeds.

• Short-Circuit Evaluation: Short-circuit evaluation means the second condition (p < q) is evaluated only if the first (x > y) is false; if x > y is already true, the code jumps immediately to L1 without ever checking p < q.

• Benefit: This minimizes unnecessary condition checking, reduces the number of comparisons executed at runtime, and avoids evaluating expressions that could be costly or that depend on conditions not yet safe to evaluate, improving overall execution efficiency.

Q5. Backpatching and Flow Control

Given:

Instruction Target

200: if x==y goto _

201: goto _

Backpatch lists:

truelist = {200}

falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Answer:

• Role of Backpatching: Backpatching lets the compiler generate conditional and unconditional jump instructions with their target addresses left temporarily blank, and fill in (patch) the correct label or address only once it becomes known later in the code.

• Truelist and Falselist: truelist = {200} holds the address of the instruction to be executed when the condition is true (if x==y goto _), and falselist = {201} holds the address for when the condition is false (goto _); both currently have their target left unresolved.

• Updating Target Addresses: When the compiler later determines the actual instruction number where the true branch (or false branch) code begins, it backpatches that address into every instruction listed in truelist (or falselist), replacing the blank target with the correct label.

• Importance in Boolean Translation: Backpatching allows Boolean expressions and control statements to be translated in a single left-to-right pass without needing to know jump targets in advance, which is essential for efficiently handling short-circuit expressions, if-else statements, and loops during intermediate code generation.

RUBRICS FOR CALCULATION

Criteria	Weightage
Understanding of Data	25%
Analysis	25%
Application of Concepts	20%
Conclusion	20%
Clarity	10%

Level 4 (Exemplary)

Understanding of Data: Accurately interprets all data patterns, trends, and relationships.

Analysis: Provides deep analysis with strong logical reasoning and justification.

Application of Concepts: Effectively applies relevant concepts/tools to interpret data accurately.

Conclusion: Draws clear, meaningful conclusions with strong insights and implications.

Clarity: Presents interpretation clearly using proper structure, visuals, and terminology.

Level 3 (Proficient)

Understanding of Data: Interprets data correctly with minor gaps.

Analysis: Provides reasonable analysis with some justification.

Application of Concepts: Applies concepts with minor errors.

Conclusion: Provides valid conclusions with moderate insights.

Clarity: Generally clear with minor issues.

Level 2 (Developing)

Understanding of Data: Partial understanding; misses key trends.

Analysis: Basic analysis with weak reasoning.

Application of Concepts: Limited application; requires guidance.

Conclusion: Conclusions are vague or weak.

Clarity: Lacks clarity or proper structure.

Level 1 (Beginning)

Understanding of Data: Misinterprets or fails to understand data.

Analysis: No meaningful analysis provided.
Application of Concepts: Unable to apply concepts to data.
Conclusion: No clear or valid conclusions.
Clarity: Poor presentation and difficult to understand.